CSA05 – DATABASE MANAGEMENT SYSTEMS

Assignment 1 – CO1

Design, Analyse and Validate a Storage and Indexing Strategy for the Museum Condition-Report Database

Item	Detail
Scenario	Meridian Museum – Collection Loans and Exhibition Planning
Nature / Duration / Marks	Individual / 1 week / 50 marks
Student name & register no.	Kayyala Vinay (192524278)
Seed used for all generated data	4211 (see Task 8)
Database engine used for the implementation	MariaDB 10.11.14 (MySQL-compatible). A MySQL server could not be installed in the sandbox used to prepare this report, so all SQL, error messages and timings in Tasks 8, 9 and 12 come from MariaDB. The DDL uses only syntax that MySQL 8.0.16+ also accepts (CHECK constraints are enforced from that version).

How to read this report

Every number labelled “measured” was produced by the scripts delivered with this report (schema.sql, generate.py, validate.py) and is stored in results.json. Statements marked **[asserted]** are design arguments that were not executed, so they should be read as business assertions and not as test results.

Design decision in one line: the preferred design is **Alternative B** – LOAN is an independently identified entity, each artifact on a loan is a LOAN_OBJECT line (the custody record), and condition evidence hangs off the LOAN_OBJECT line, so it is filed by physical object and not by lender name. For the changed requirement (Task 12) the design is **modified**, not redesigned: a dated ARTIFACT_OWNERSHIP association is added and custody (LOAN_OBJECT) stays untouched.

Task 1 – Requirements Analysis and Three Alternative Conceptual Designs (5 marks)

1.1 Requirement analysis

Rule in the case	Modelling consequence
Same artifact has different descriptions in loan register and gallery plan	One ARTIFACT entity holds the catalog description; loan register and gallery plan both reference it by artifact_id and never copy the text.
Loan expiry is not linked to display slots	Placement carries its own dates and must be checkable against the loan period (cross-table rule, Task 8).
Condition reports are filed by lender name	Condition evidence must be keyed by the physical object on a loan line (loan_id, artifact_id), not by lender.
8 units, 35 curators, 1,200 lenders, 9,000 artifacts	Scale basis for generated data (ratios kept in Task 8).
Each artifact has one current lender; a loan belongs to one lender and holds many artifacts	LENDER 1:N LOAN, LOAN 1:N LOAN_OBJECT, ARTIFACT 1:N LOAN_OBJECT (successive loans).
Display slots are numbered within an exhibition	DISPLAY_SLOT is a weak entity keyed (exhibition_id, slot_no).
A curator may lend a private object	Roles overlap: a person can be both CURATOR and LENDER → shared PERSON identity.
Ownership, custody, placement and condition must be separate	Four separate constructs: ARTIFACT_OWNERSHIP (Task 12), LOAN/LOAN_OBJECT, PLACEMENT/DISPLAY_SLOT, CONDITION_REPORT.

Operational users who create LOAN records. Registrars create the loan (lender, dates, insured values); curators add condition reports and plan exhibitions; lender-relations staff maintain contact details. The information they must share is: lender identity, artifact identity (accession number), loan period, per-artifact insured value and custody dates, and the condition rating at hand-over.

1.2 The three alternatives

Alternative A – Dated M:N relationship (LENDER – ARTIFACT)

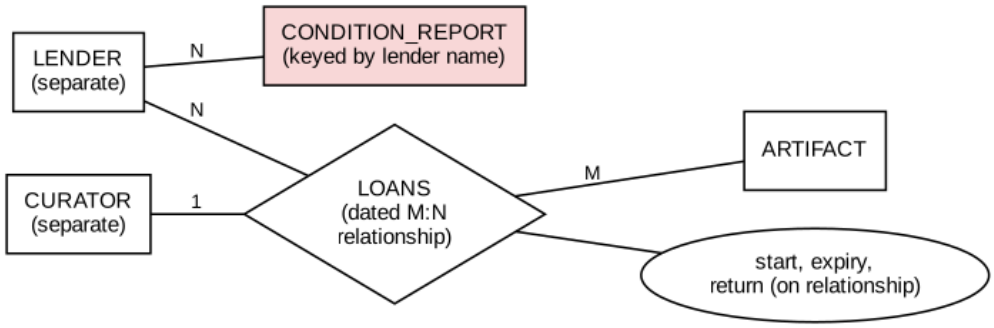


Figure 1 – Alternative A: loan as a dated relationship; separate LENDER and CURATOR; reports filed by lender

- **Identity:** the loan has no identity of its own; it is the pair (lender, artifact) plus a start date.
- **Degree:** binary LENDER–ARTIFACT with a CURATOR link.
- **History:** dates are relationship attributes.
- **Preserves custody and condition?** Only if start_date is part of the relationship key; condition reports are attached to the lender, so evidence is lost when two lenders share a name.
- **Failure case it resolves:** the simple “who lent what” question, with minimal tables.
  Counterexample from this case: lender Iyer lends artifact 609 in 2025 and again in 2026 – key (lender, artifact) collides unless the date is added, and a report filed under “Iyer” cannot say which loan it belongs to.

Alternative B – Independently identified LOAN with LOAN_OBJECT lines (preferred)

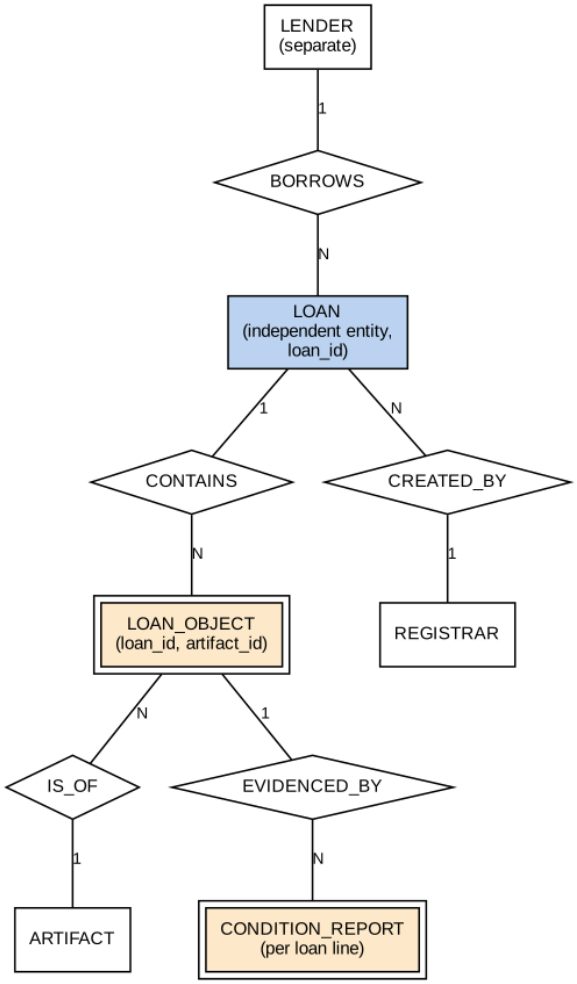


Figure 2 – Alternative B: LOAN (loan_id), LOAN_OBJECT (custody line), CONDITION_REPORT per line

- **Identity:** LOAN has its own loan_id; LOAN_OBJECT is identified by (loan_id, artifact_id).
- **Degree:** binary relationships only; the M:N between LOAN and ARTIFACT is resolved by the LOAN_OBJECT line, which owns custody dates, insured value and condition reports.
- **History:** every loan occurrence is a separate record, so successive loans of the same artifact are separate rows.
- **Preserves custody and condition:** custody_from/custody_to sit on the line, and CONDITION_REPORT references the line with a composite foreign key – evidence cannot exist without the borrowed object.
- **Failure case it resolves:** one loan with four artifacts where one is returned early – per-line custody_to records that; reports stay attached to that line.

Alternative C – Ternary LOAN_EVENT among PERSON, CURATOR and EXHIBITION

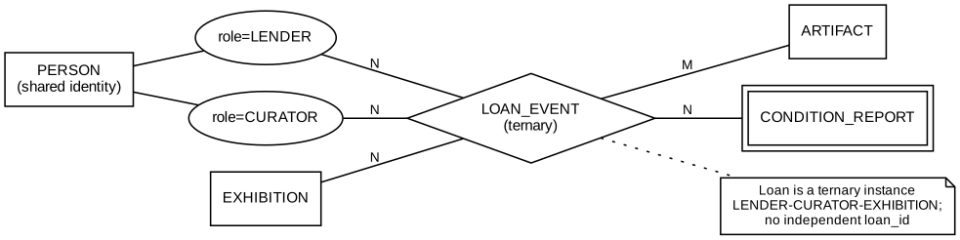


Figure 3 – Alternative C: ternary loan event with shared PERSON identity

- **Identity:** the loan is the tuple (lender-person, curator-person, exhibition); there is no loan_id.
- **Degree:** ternary.

- **History:** a new tuple per exhibition.
- **Failure case it resolves:** a curator who lends a private object – with shared PERSON identity there is a single person record playing both roles. **Its own failure:** a loan registered before the exhibition is decided cannot be stored, because every tuple needs all three participants.

1.3 Comparison on the required dimensions

Dimension	A – dated relationship	B – independent LOAN (chosen)	C – ternary + PERSON
Identity	Composite of participants + date	Own surrogate loan_id + loan_ref	Composite of three participants
Relationship degree	Binary	Binary (M:N resolved)	Ternary
Historical representation	Date attributes on the relationship	One row per loan occurrence, dates on loan and line	One tuple per event
LENDER vs employee records	Separate (duplicates the curator-lender)	Separate in the base model; merged by EER in Task 5	Shared PERSON from the start
Where condition evidence lives	On lender (wrong)	On LOAN_OBJECT line	On the ternary event
Custody periods	Weak – start date only	Strong – per line	Medium – per event, not per object

Why renaming LOAN would leave the same model. Renaming the relationship in Alternative A from LOANS to LOAN changes only the label. Its identity is still derived from LENDER and ARTIFACT keys, its degree is still binary, and its history is still attributes on the relationship, so the same collision (same lender, same artifact, second loan) remains. A different model needs a different identity (B), a different degree (C), or a different historical treatment.

Dated relationship versus independent LOAN occurrence. In A the facts “lender L lent artifact X starting 2025-03-01” cannot be referred to by other records – a condition report has nothing to point at except lender+artifact+date. In B the loan occurrence is an entity that other records (LOAN_OBJECT, CONDITION_REPORT, PLACEMENT) reference through loan_id.

Separate LENDER and employee records versus shared PERSON. Separate records mean that a curator lending a private object appears twice with possibly different phone numbers; the condition evidence signed by the curator and the loan lent by the same person cannot be tied together. Shared PERSON stores the identity once and adds role subtypes (Task 5).

Choice. B is chosen because it is the only alternative that gives a stable reference for custody and condition evidence without losing loans that are registered before exhibitions are planned. The PERSON idea from C is adopted inside B at the EER stage.

Task 2 – Entities, Attributes and Keys (2 marks)

2.1 Entity and key catalog

Entity	Type	Primary key	Candidate / business keys	Why
PERSON	Strong	person_id	email (unique)	Names repeat; person_id is stable.
LENDER, CURATOR, REGISTRAR	Subtype	person_id (inherited)	–	Role records share PERSON identity.
COLLECTION_UNIT	Strong	unit_id	unit_name	8 units; name is unique but may be renamed.
COLLECTION_TYPE	Strong (coded)	type_code	type_name	Has its own attribute (handling_note).

Entity	Type	Primary key	Candidate / business keys	Why
ARTIFACT	Strong	artifact_id	accession_no	accession_no is the business key; catalog_title is not unique.
LOAN	Strong	loan_id	loan_ref	loan_ref is what staff quote; loan_id never changes.
LOAN_OBJECT	Associative	(loan_id, artifact_id)	–	An artifact appears once per loan.
CONDITION_REPORT	Dependent	(loan_id, artifact_id, report_no)	–	Exists only for a loan line.
EXHIBITION	Strong	exhibition_id	none reliable	Names are reused (generated data contains repeated names).
DISPLAY_SLOT	Weak	(exhibition_id, slot_no)	–	slot_no is numbered within an exhibition.
PLACEMENT	Associative	(loan_id, artifact_id)	(exhibition_id, slot_no, placed_from)	One placement per loan line; a slot cannot start two placements on the same day.
ARTIFACT_OWNERSHIP	Dependent (Task 12)	(artifact_id, owner_id, valid_from)	–	A share held from a date.

Real-world identity versus record labels. accession_no and loan_ref are labels people write on paper; person_id, artifact_id and loan_id are the identities the database relies on. Labels can be mistyped or reissued, so they are UNIQUE business keys, not primary keys.

Can an exhibition name identify a display slot? No. Exhibition names are not unique (the generated data deliberately reuses them, as museums reuse titles such as “Treasures of the South”), and slot numbers restart for each exhibition. “Slot 7” is only meaningful with an exhibition identifier, so the identifying key is (exhibition_id, slot_no) with exhibition_id as the owner part.

Why a name is not a reliable identifier. In the generated data of 100 lenders there are 16 first names and 12 surnames, so duplicates are certain; a lender called “Reddy” is not a unique person. The condition-report-by-lender-name problem in the case is exactly this error.

2.2 Attribute classification

Attribute	Classification	Treatment
Contact information – phone numbers	Multivalued	Separate table PERSON_PHONE(person_id, phone_no).
Contact information – e-mail	Simple, single-valued	Column in PERSON, unique.
Name	Composite	first_name, last_name stored separately.
CollectionType	Coded concept with own data	Entity COLLECTION_TYPE (type_code, type_name, handling_note).
start_date, expiry_date, return_date, custody_from/to	Simple stored (source dates)	Stored on LOAN / LOAN_OBJECT.
Loan duration, overdue flag, artifacts per loan, report count	Derived (calculated summaries)	Not stored; computed in queries/views (Q1, Q3).
status, stage, rating	Simple, small closed domain	ENUM column; no own data, so no entity needed.

2.3 Key selection

- **Stable primary keys:** person_id, artifact_id, loan_id, exhibition_id, unit_id (AUTO_INCREMENT integers that never change meaning).
- **Business candidate keys:** accession_no, loan_ref, email, unit_name – enforced with UNIQUE constraints.
- **Owner-scoped key:** DISPLAY_SLOT (exhibition_id, slot_no).

Concurrent registration without duplicate identifiers. Surrogate keys use AUTO_INCREMENT, so InnoDB hands each concurrent insert a distinct value under its auto-increment lock. Business labels (loan_ref, accession_no) are protected by UNIQUE indexes: if two registrars type the same loan_ref at once, one INSERT succeeds and the other fails with error 1062, as demonstrated in Task 8 for a duplicate loan_ref. For slot_no, the registering transaction must lock the exhibition row (SELECT … FOR UPDATE) and read MAX(slot_no)+1; the composite primary key still rejects a collision. **[asserted – a concurrent-session test was not run.]**

Task 3 – Relationships and Structural Constraints (6 marks)

3.1 Relationship catalog with participation

Notation: (min,max) is read at the entity next to the bracket – how many relationship instances one entity occurrence takes part in.

Relationship	Left entity	Right entity	Justification from the case
LENDER – BORROWS – LOAN	LENDER (0,N)	LOAN (1,1)	A loan belongs to exactly one lender; a lender may have no loan yet.
LOAN – CONTAINS – LOAN_OBJECT	LOAN (1,N)	LOAN_OBJECT (1,1)	A loan contains multiple artifacts; a line belongs to one loan. The “at least one line” rule needs application/trigger logic (Task 8).
LOAN_OBJECT – IS_OF – ARTIFACT	LOAN_OBJECT (0,N from artifact side)	ARTIFACT (1,1)	An artifact can have successive loans; each line is for one artifact.
LOAN_OBJECT – EVIDENCED_BY – CONDITION_REPORT	LOAN_OBJECT (0,N)	CONDITION_REPORT (1,1)	Reports are optional per line (measured: lines without any report exist, Q2) but a report cannot exist without its line.
EXHIBITION – HAS_SLOT – DISPLAY_SLOT	EXHIBITION (0,N)	DISPLAY_SLOT (1,1)	Identifying relationship; weak entity total participation.
LOAN_OBJECT – PLACED_IN – DISPLAY_SLOT	LOAN_OBJECT (0,1)	DISPLAY_SLOT (0,N over time)	A line may not be on display; the slot reused by dated placements. Composite FK (exhibition_id, slot_no).
ARTIFACT – OWNED_BY – PERSON	ARTIFACT (1,N)	PERSON (0,N)	Task 12: shared ownership; at least one owner at any time.
LOAN – CREATED_BY – REGISTRAR	LOAN (1,1)	REGISTRAR (0,N)	Every loan is created by one registrar.

3.2 Four relationships with explicit bounds

Relationship	Min/max	Custody / condition effect
LOAN CONTAINS LOAN_OBJECT	LOAN (1,N); LOAN_OBJECT (1,1)	Custody is recorded only on lines; a loan without lines would have no custody.
LOAN_OBJECT EVIDENCED_BY CONDITION_REPORT	LOAN_OBJECT (0,N); CONDITION_REPORT (1,1)	Missing evidence is allowed by the schema and must be found by query (Q2); with the DDL, a report cannot reference a missing line.
EXHIBITION HAS_SLOT DISPLAY_SLOT	EXHIBITION (0,N); DISPLAY_SLOT (1,1)	Deleting an exhibition removes its slots (ON DELETE CASCADE).
LENDER BORROWS LOAN	LENDER (0,N); LOAN (1,1)	A lender with no loans is valid; a loan with no lender is not.

3.3 Which links are compulsory; which parents may exist alone

Compulsory (NOT NULL foreign keys in the DDL): LOAN → lender, registrar; LOAN_OBJECT → loan and artifact; CONDITION_REPORT → loan line and curator; ARTIFACT → current lender, type, unit. Parents that may exist without dependants: LENDER without loans, ARTIFACT without loans, EXHIBITION without slots, LOAN_OBJECT without condition reports or placement. Measured at N =

100: 106 loans exist for 101 lenders, 55 loan-line rows have no report, and 110 of 249 lines have a placement.

3.4 Binary or ternary: LENDER, CURATOR, EXHIBITION

Three binary relationships (lender–loan, curator–exhibition, loan–exhibition) say who lends, who curates and what is exhibited, but not that **this lender's object was handled by this curator for this exhibition**. A ternary relationship is needed only if that three-way fact cannot be rebuilt from the binary facts. The test below shows it cannot.

Ternary instance and projection test

Ternary tuple (LENDER, CURATOR, EXHIBITION)
(L1, C1, E1)
(L1, C2, E2)
(L2, C1, E2)

Projection	Pairs
LENDER–CURATOR	(L1,C1), (L1,C2), (L2,C1)
CURATOR–EXHIBITION	(C1,E1), (C2,E2), (C1,E2)
LENDER–EXHIBITION	(L1,E1), (L1,E2), (L2,E2)

Natural join of the three projections gives four triples: (L1,C1,E1), (L1,C2,E2), (L2,C1,E2) **and the extra tuple (L1,C1,E2)**, because L1–C1, C1–E2 and L1–E2 all exist as pairs. The extra tuple claims that lender L1's object was handled by curator C1 in exhibition E2, which never happened. For custody and condition evidence this means a fabricated custody period and fabricated responsibility for a condition report. The ternary fact is therefore not reducible to binaries – **but** in Meridian's final design the loan is itself an entity (LOAN), so the three-way fact is carried by LOAN_OBJECT, PLACEMENT and CONDITION_REPORT references, avoiding a ternary relationship. **[the projection test was worked by hand on a small instance, not in SQL]**

3.5 Where dated or quantitative facts belong

Fact	Placed on	Reason
start_date, expiry_date, return_date	LOAN	Agreed once with the lender for the whole loan.
custody_from, custody_to	LOAN_OBJECT	One object can come back earlier than the rest of the loan.
insured_value	LOAN_OBJECT	A loan-line valuation, not an artifact value: the same artifact can be insured differently on successive loans.
placed_from, placed_to, slot	PLACEMENT	Depends on the line and the slot.
share_pct, valid_from, valid_to	ARTIFACT_OWNERSHIP	Depends on both artifact and owner, over time.

Effect of missing LOAN_OBJECT evidence. If a loan line has no condition report the model does not break, but the museum cannot prove the object's state at hand-over. This is a data-completeness issue that structural cardinality (0,N) allows and that query Q2 detects (Task 9).

Task 4 – Chen ER Diagram (4 marks)

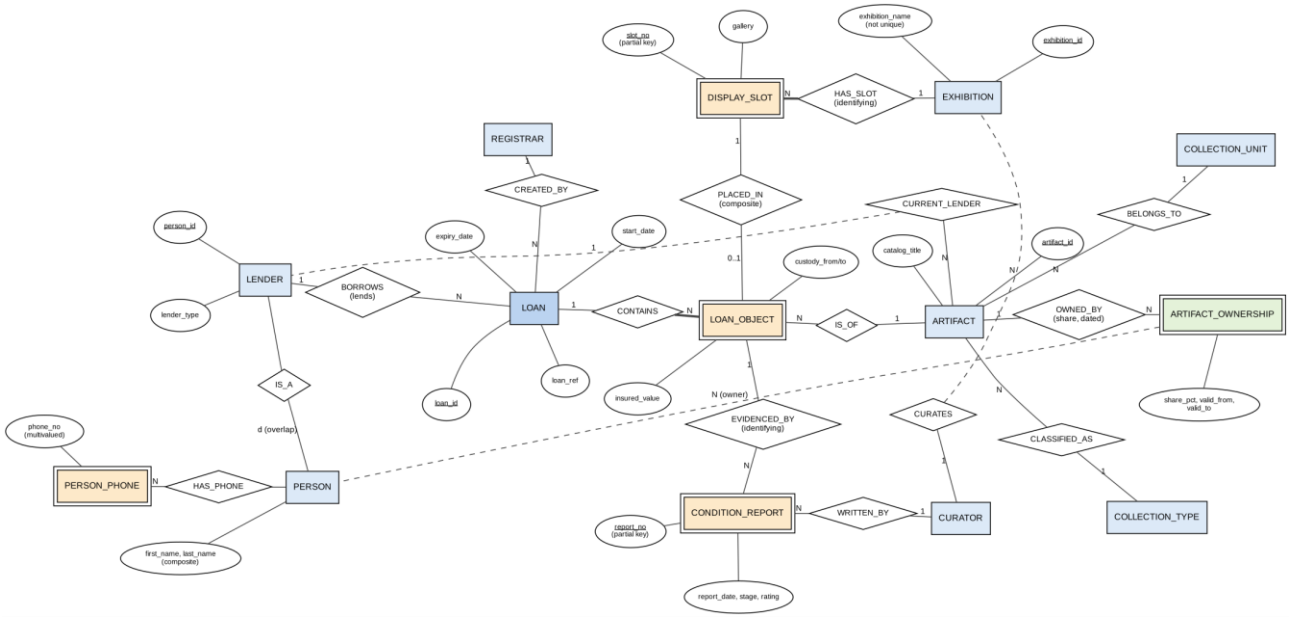


Figure 4 – Chen ER diagram of the preferred model (Alternative B), including ARTIFACT_OWNERSHIP from Task 12

Legend

Symbol	Meaning
Single rectangle	Strong entity (own key)
Double rectangle	Weak or dependent entity (existence depends on an owner)
Diamond	Relationship; “identifying” marks an owner–weak link
Ellipse; underlined text	Attribute; key attribute (partial key for weak entities)
Thick line	Total (compulsory) participation – LOAN contains lines, slot belongs to exhibition
1, N, 0..1 near an entity	Cardinality seen from that entity
Dashed line	Repeated reference to an entity drawn elsewhere (not a new entity)

Repeated references. PERSON is drawn once; LENDER reaches it through IS_A, and ARTIFACT_OWNERSHIP points back to it with a dashed line instead of a second PERSON box. CURRENT_LENDER (ARTIFACT → LENDER) and CURATES (CURATOR → EXHIBITION) are dashed for the same reason: the entity boxes already exist and duplicating them would obscure the custody path LOAN → LOAN_OBJECT → CONDITION_REPORT.

Separate paths. Borrowed objects: LENDER → LOAN → LOAN_OBJECT → ARTIFACT. Condition reports: LOAN_OBJECT → CONDITION_REPORT → CURATOR. Placement: LOAN_OBJECT → DISPLAY_SLOT → EXHIBITION. Ownership: ARTIFACT → ARTIFACT_OWNERSHIP → PERSON.

Attribute catalog

Entity	Attributes (key underlined in words as PK)	Notes
PERSON	PK person_id; first_name, last_name; email	name composite; phones multivalued
PERSON_PHONE	PK (person_id, phone_no)	multivalued attribute entity
LENDER	PK person_id; lender_type	subtype of PERSON
CURATOR	PK person_id; unit_id; hire_date	subtype
REGISTRAR	PK person_id; certification	subtype
COLLECTION_UNIT	PK unit_id; unit_name	8 rows
COLLECTION_TYPE	PK type_code; type_name; handling_note	

Entity	Attributes (key underlined in words as PK)	Notes
ARTIFACT	PK artifact_id; accession_no; catalog_title; type_code; unit_id; current_lender_id	one catalog description only
LOAN	PK loan_id; loan_ref; lender_id; registrar_id; start_date; expiry_date; return_date; status	
LOAN_OBJECT	PK (loan_id, artifact_id); insured_value; custody_from; custody_to	custody record
CONDITION_REPORT	PK (loan_id, artifact_id, report_no); report_date; stage; rating; curator_id	evidence
EXHIBITION	PK exhibition_id; exhibition_name; open_date; close_date; curator_id	name not unique
DISPLAY_SLOT	PK (exhibition_id, slot_no); gallery	weak
PLACEMENT	PK (loan_id, artifact_id); exhibition_id; slot_no; placed_from; placed_to	composite FK to slot
ARTIFACT_OWNERSHIP	PK (artifact_id, owner_id, valid_from); valid_to; share_pct	Task 12

The identifying owner of DISPLAY_SLOT is EXHIBITION; its owner-scoped key is (exhibition_id, slot_no). Compulsory participation (thick lines) and cardinalities agree with Tasks 2 and 3.

Task 5 – EER Model (3 marks)

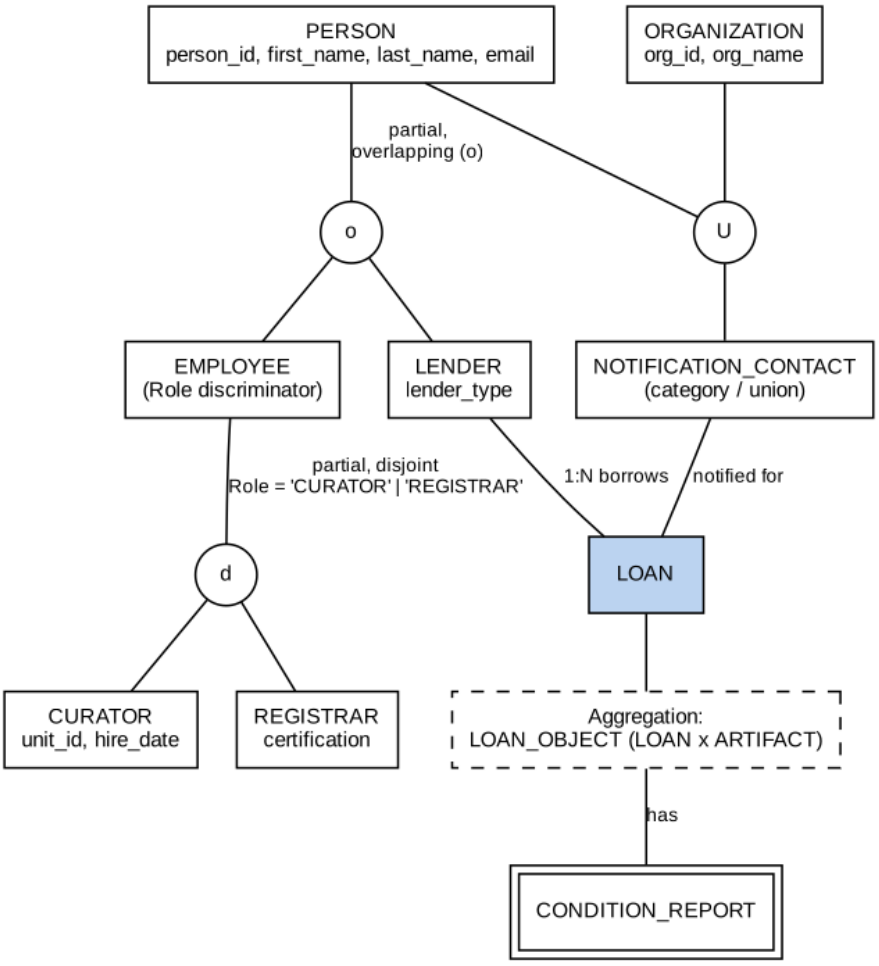


Figure 5 – EER: PERSON generalization, role specialization, union category and aggregation

- **Generalization:** LENDER and employees share name, e-mail and phones, so they are generalized to PERSON. EMPLOYEE is specialized into CURATOR and REGISTRAR.
- **Overlap and coverage:** the LENDER/EMPLOYEE specialization is **overlapping** (o) – a curator who lends a private object is both – and **partial**, because a PERSON may be a contact only. Measured: at N = 100, 1 curator is also a lender; at 1,000 there are 5; at 10,000, 42; at 100,000,

417 (query Q7). The CURATOR/REGISTRAR specialization is **disjoint** (d) and **partial** so other employee roles can be added later.

- **Role discriminator:** the employee attribute Role ∈ {CURATOR, REGISTRAR} decides subtype membership (attribute-defined specialization). For custody this decides who may write a condition report (CURATOR) and who may create a LOAN (REGISTRAR).
- **Union category:** NOTIFICATION_CONTACT is a category of PERSON ∪ ORGANIZATION – a notification for an expiring loan may go to a person or to an institution, and the contact has a single identity regardless of its source. It is shown at EER level only and is not implemented in the DDL used for Tasks 8–9.
- **Aggregation:** the pair LOAN × ARTIFACT (LOAN_OBJECT) is treated as one unit that has condition reports, so the report relates to the aggregate rather than to the loan or artifact alone.

Subtypes versus one wide record

Aspect	Subtypes under PERSON	One wide record
Curator who lends a private object	One PERSON row, plus a LENDER row and a CURATOR row	One row with every column of all roles filled
NULLs	None for role-specific fields	hire_date, certification, lender_type mostly NULL
Role-specific constraints	Foreign keys target the right subtype (LOAN→REGISTRAR, CONDITION_REPORT→CURATOR)	FK cannot say “only curators”; needs CHECK/trigger
Cost	One extra join to read role attributes	No join, but wide rows and weak integrity

Task 6 – Weak Entities and Complex Attributes (3 marks)

- **DISPLAY_SLOT:** owner EXHIBITION, local (partial) key slot_no, full key (exhibition_id, slot_no). The link is an **identifying** relationship: exhibition_id is part of the primary key and the foreign key is ON DELETE CASCADE. By contrast, ARTIFACT.current_lender_id is an **ordinary foreign key**: it is not part of ARTIFACT’s key, and a lender must not be deleted while artifacts reference it (measured error 1451, Task 8).
- **Multivalued phone numbers:** PERSON_PHONE(person_id, phone_no) holds one row per number. A comma-separated column would prevent searching and indexing one number and cannot be validated; with custody, it would make notifying a lender at the number used on a given loan impossible to audit. Measured: 145 phone rows for 108 persons at N = 100.
- **Composite name:** first_name and last_name are stored separately so sorting and matching are possible; a full name is composed on output.
- **Derived values:** loan duration (expiry_date – start_date), overdue status (status = OPEN and expiry_date < current date), number of artifacts on a loan and number of reports per line are computed, not stored – stored copies would go stale when a custody_to or report is added. Custody-period and condition summaries are computed in Q1, Q2 and Q3.
- **Coded concepts needing entity identity:** COLLECTION_TYPE has handling_note, COLLECTION_UNIT has its own curators. Concepts that stay as ENUM: loan status, report stage and rating, because they have no data of their own.
- **Artifact value versus loan-line valuation:** the schema has no “value” on ARTIFACT; the insured_value sits on LOAN_OBJECT because the same object can be valued differently on successive loans.

Task 7 – Mapping the EER Model to a Relational Schema (4 marks)

7.1 Mapping steps

Step	Construct	Result
1	Strong entities	PERSON, COLLECTION_UNIT, COLLECTION_TYPE, ARTIFACT, LOAN, EXHIBITION, each with its primary key.
2	Weak entity	DISPLAY_SLOT (exhibition_id, slot_no) with FK to EXHIBITION; CONDITION_REPORT with composite FK to LOAN_OBJECT.
3	1:1 associations	None in the model. (Subtype–supertype shares the primary key person_id.)
4	1:N associations	Foreign keys on the N side: loan.lender_id, loan.registrar_id, artifact.current_lender_id, artifact.type_code, artifact.unit_id, exhibition.curator_id.
5	M:N associations	LOAN × ARTIFACT → LOAN_OBJECT (with insured_value and custody dates); LOAN_OBJECT × DISPLAY_SLOT → PLACEMENT; ARTIFACT × PERSON → ARTIFACT_OWNERSHIP.
6	Multivalued attribute	PERSON_PHONE.
7	Specialization	Option 8A: one relation for PERSON and one per subtype sharing person_id as PK/FK.

7.2 Specialization option choice

Option	Verdict	Reason
8A – superclass + one relation per subclass	**Chosen**	Works for overlapping and partial specialization; a curator who lends has one PERSON row and two subtype rows; FKs can target LENDER, CURATOR or REGISTRAR exactly.
8B – subclass relations only	Rejected	A person who is only a contact would have no home; duplicates shared attributes for overlapping persons.
8C – single relation with type attribute	Rejected	Cannot represent overlap (curator and lender) with a single type value.
8D – single relation with one flag per role	Rejected	Works for overlap but yields a wide table with many NULLs and no FK targeting.

7.3 Schema catalog

Relation	PK	Foreign keys (ON DELETE)
person	person_id	–
person_phone	(person_id, phone_no)	person_id → person (CASCADE)
lender	person_id	person_id → person (RESTRICT)
curator	person_id	person_id → person; unit_id → collection_unit
registrar	person_id	person_id → person
collection_unit	unit_id	–
collection_type	type_code	–
artifact	artifact_id	type_code → collection_type; unit_id → collection_unit; current_lender_id → lender
loan	loan_id	lender_id → lender; registrar_id → registrar
loan_object	(loan_id, artifact_id)	loan_id → loan; artifact_id → artifact
condition_report	(loan_id, artifact_id, report_no)	(loan_id, artifact_id) → loan_object; curator_id → curator
exhibition	exhibition_id	curator_id → curator
display_slot	(exhibition_id, slot_no)	exhibition_id → exhibition (CASCADE)
placement	(loan_id, artifact_id)	(loan_id, artifact_id) → loan_object; (exhibition_id, slot_no) → display_slot
artifact_ownership	(artifact_id, owner_id, valid_from)	artifact_id → artifact; owner_id → person

This catalog was written from the same DDL (appendix) that ran in Tasks 8–9, so every reference agrees with the implementation. All foreign keys other than person_phone and display_slot use the default RESTRICT.

Deletion behaviour for historical loans. LOAN, LOAN_OBJECT, CONDITION_REPORT and PLACEMENT use RESTRICT: historical custody and condition evidence must not disappear because a parent is deleted. Measured: deleting a lender referenced by an artifact fails with error 1451. A loan that should no longer count is marked status = CANCELLED instead of deleted. Display slots and phone numbers are CASCADE because they have no meaning without the owner. A mapping effect is that an exhibition with placements cannot be deleted without first removing its placements **[asserted from the DDL; not executed]**.

Task 8 – Implementation and Instances (4 marks)

8.1 Reproducible loading procedure

Step	Command / action
1	Start MariaDB/MySQL; create user with rights to create databases.
2	python3 generate.py – for each size N in {100, 1000, 10000, 100000} it drops/creates database meridian_N, runs schema.sql, and populates it with random.Random(4211 + N) (seed 4211 combined with the size so that each size is reproducible alone).
3	python3 validate.py – runs the eight queries (median of 7 runs), the JOIN counts, the invalid writes, the flat-file anomalies and the Task 12 checks, and writes results.json.

Scaling rules. Lenders = N (plus curators who also lend); artifacts = 7.5 × N (the case ratio 9,000 / 1,200); curators = max(5, 35N/1200); registrars = max(3, N/200); collection units fixed at 8; exhibitions = max(4, N/25) with 30 slots each.

How source keys are selected for linked rows. Parents are inserted first. Each artifact picks its current lender uniformly from all lenders. For each lender with artifacts, 0–2 loans are generated (weights 0, 1, 1, 2), and each loan takes 1–4 distinct artifacts **of that lender only**, so a loan's lines always belong to its lender. About 80 % of lines receive an INCOMING report (plus an OUTGOING report if returned). Placements consume slots from a shuffled pool without replacement, so no slot is reused. Overdue loans occur because 30 % of loans past expiry are left OPEN. Ownership: 70 % of artifacts have one 100 % owner, 30 % have two owners whose shares changed from 50/50 to 60/40 on 2026-01-01.

8.2 Four stable-schema populations (measured row counts)

Table	N = 100	N = 1,000	N = 10,000	N = 100,000
person	108	1,034	10,342	103,417
lender	101	1,005	10,042	100,417
curator	5	29	292	2,917
registrar	3	5	50	500
artifact	750	7,500	75,000	750,000
loan	106	986	10,046	99,747
loan_object	249	2,435	24,724	246,623
condition_report	294	2,941	29,690	292,860
exhibition	4	40	400	4,000
display_slot	120	1,200	12,000	120,000
placement	110	1,200	12,000	120,000
artifact_ownership	1,440	14,157	142,716	1,424,334

Table	N = 100	N = 1,000	N = 10,000	N = 100,000
person_phone	145	1,358	13,752	137,801

The same 15 relations exist at every size; only the instances change. Ratios are proportionate (artifacts ≈ 7.5 per lender, loans ≈ 1 per lender, loan lines ≈ 2.5 per loan).

8.3 Invalid writes and the actual failure messages (measured on meridian_100)

Attempt	Actual message (MariaDB)
Duplicate identity (loan_ref) INSERT INTO loan(loan_ref,lender_id,registrar_id,start_date,expiry_date,status) VALUES ('LN-0000001',1,107,'20…	IntegrityError: 1062 - Duplicate entry 'LN-0000001' for key 'uq_loan_ref'
Duplicate composite key (loan line) INSERT INTO loan_object SELECT loan_id,artifact_id,100,start_date,NULL FROM (SELECT lo.loan_id,lo.artifact_id,…	IntegrityError: 1062 - Duplicate entry '1-174' for key 'PRIMARY'
Invalid loan period (expiry before start) INSERT INTO loan(loan_ref,lender_id,registrar_id,start_date,expiry_date,status) VALUES ('LN-BAD01',1,107,'2026…	OperationalError: 4025 - CONSTRAINT `ck_loan_period` failed for `meridian_100`.`loan`
Workflow: bad status value INSERT INTO loan(loan_ref,lender_id,registrar_id,start_date,expiry_date,status) VALUES ('LN-BAD02',1,107,'2026…	DataError: 1265 - Data truncated for column 'status' at row 1
Category: unknown collection type INSERT INTO artifact(accession_no,catalog_title,type_code,unit_id,current_lender_id) VALUES ('ACC-X1','Test','…	IntegrityError: 1452 - Cannot add or update a child row: a foreign key constraint fails (`meridian_100`.`artifact`, CONSTRAINT `fk_art_type` FOREIGN KEY (`type_code`) REFERENCES `collection_type` (`type_code`))
Missing parent: loan for non-existent lender INSERT INTO loan(loan_ref,lender_id,registrar_id,start_date,expiry_date,status) VALUES ('LN-BAD03',99999,107,'…	IntegrityError: 1452 - Cannot add or update a child row: a foreign key constraint fails (`meridian_100`.`loan`, CONSTRAINT `fk_loan_lender` FOREIGN KEY (`lender_id`) REFERENCES `lender` (`person_id`))
Weak entity: slot for missing exhibition INSERT INTO display_slot VALUES (9999,1,'Gallery 1')	IntegrityError: 1452 - Cannot add or update a child row: a foreign key constraint fails (`meridian_100`.`display_slot`, CONSTRAINT `fk_slot_ex` FOREIGN KEY (`exhibition_id`) REFERENCES `exhibition` (`exhibition_id`) ON DELETE CASCADE)
Composite FK: placement into missing slot INSERT INTO placement VALUES (2,3,1,999,'2026-01-01','2026-02-01')	IntegrityError: 1452 - Cannot add or update a child row: a foreign key constraint fails (`meridian_100`.`placement`, CONSTRAINT `fk_pl_slot` FOREIGN KEY (`exhibition_id`, `slot_no`) REFERENCES `display_slot` (`exhibition_id`, `slot_no`))
Negative insured value INSERT INTO loan_object VALUES (1,1,-5,'2026-01-01',NULL)	OperationalError: 4025 - CONSTRAINT `ck_lo_value` failed for `meridian_100`.`loan_object`
Ownership share over 100 INSERT INTO artifact_ownership VALUES (1,1,'2027-01-01',NULL,150)	OperationalError: 4025 - CONSTRAINT `ck_own_share` failed for `meridian_100`.`artifact_ownership`
Delete lender referenced by loans DELETE FROM lender WHERE person_id=(SELECT lender_id FROM loan LIMIT 1)	IntegrityError: 1451 - Cannot delete or update a parent row: a foreign key constraint fails (`meridian_100`.`artifact`, CONSTRAINT `fk_art_lender` FOREIGN KEY (`current_lender_id`) REFERENCES `lender` (`person_id`))

Note on the composite-FK attempt. My first attempt to violate the placement→display_slot foreign key reused an existing primary key, so the server reported a duplicate key (1062) before the foreign key was tested. The row above is the re-run with a loan line that had no placement; it proves the composite constraint fk_pl_slot.

Another observation: the invalid status value “ARCHIVED” gave “Data truncated” (1265) rather than a CHECK message, because the value is rejected at the ENUM type itself before any constraint is evaluated.

8.4 Rules that need more than a row-level CHECK

Rule	Why CHECK is not enough	Evidence
Current ownership shares of an artifact total 100 %	Needs aggregation across rows of ARTIFACT_OWNERSHIP	Measured: inserting a 30 % share for an artifact already at 100 % was **accepted**; the total became 130.00 (then removed). Q8 detects it.
A placement must lie inside the loan period	Compares PLACEMENT dates with LOAN dates in another table	Measured: updating placed_to to 2099-01-01 was **accepted**; Q6 then returned 1 violating row (then repaired).
A loan must have at least one line	Needs a count over LOAN_OBJECT after the transaction	**[asserted]** requires a trigger/deferred check; not implemented.
One artifact cannot be in two overlapping custody periods	Overlap test across rows	**[asserted]**; not implemented.
Condition report date lies inside custody dates	Compares CONDITION_REPORT with LOAN_OBJECT	**[asserted]**; not implemented.

Row-level CHECK constraints that did work: expiry_date > start_date (ck_loan_period), insured_value ≥ 0, share_pct between 0 and 100, slot_no > 0.

Task 9 – Validation with Use-Case Queries (3 marks)

9.1 The eight queries

```
-- Q1 Overdue loans still open
SELECT loan_id, loan_ref, lender_id, expiry_date FROM loan WHERE status='OPEN' AND expiry_date <
'2026-11-05';
-- Q2 Artifacts (on a loan line) with no condition evidence
SELECT a.artifact_id, a.accession_no FROM artifact a
  JOIN loan_object lo ON lo.artifact_id=a.artifact_id
  LEFT JOIN condition_report cr ON cr.loan_id=lo.loan_id AND cr.artifact_id=lo.artifact_id
  WHERE cr.report_no IS NULL;
-- Q3 Loans with 3+ artifacts
SELECT loan_id, COUNT(*) n FROM loan_object GROUP BY loan_id HAVING COUNT(*)>=3;
-- Q4 Lenders with more than one loan
SELECT lender_id, COUNT(*) n FROM loan GROUP BY lender_id HAVING COUNT(*)>1;
-- Q5 Exhibition inventory through placements
SELECT e.exhibition_id, s.slot_no, a.accession_no FROM exhibition e
  JOIN display_slot s USING(exhibition_id)
  JOIN placement p ON p.exhibition_id=s.exhibition_id AND p.slot_no=s.slot_no
  JOIN artifact a ON a.artifact_id=p.artifact_id;
-- Q6 Placements outside the loan period (empty = good)
SELECT p.loan_id,p.artifact_id FROM placement p JOIN loan l ON l.loan_id=p.loan_id
  WHERE p.placed_to > l.expiry_date OR p.placed_from < l.start_date;
-- Q7 Curators who also lend
SELECT c.person_id FROM curator c JOIN lender le ON le.person_id=c.person_id;
-- Q8 Artifacts whose current ownership shares do not total 100
SELECT artifact_id, SUM(share_pct) FROM artifact_ownership WHERE valid_to IS NULL
  GROUP BY artifact_id HAVING SUM(share_pct)<>100;
```

9.2 Actual output (first rows at N = 100; full counts in 9.3)

Query	Actual output rows (first 5 shown)
Q1	(60, LN-0000060, 33, 2025-08-24) (94, LN-0000094, 69, 2025-10-12) (52, LN-0000052, 39, 2025-10-23) (64, LN-0000064, 50, 2025-11-09) (84, LN-0000084, 94, 2025-11-21)
Q2	(28, ACC-0000028) (29, ACC-0000029) (39, ACC-0000039) (48, ACC-0000048) (49, ACC-0000049)
Q3	(2, 4) (3, 4) (6, 4) (7, 4) (15, 4)
Q4	(7, 2) (9, 2) (10, 2) (11, 2) (13, 2)
Q5	(4, 1, ACC-0000731) (4, 2, ACC-0000332) (4, 3, ACC-0000419) (4, 4, ACC-0000500) (4, 5, ACC-0000709)
Q6	**Empty set** – no placement lies outside its loan period (at all four sizes). This is the expected result for a clean load; it became 1 after the deliberate bad update in Task 8.
Q7	(101)
Q8	**Empty set** – every artifact's current shares total 100 (at all four sizes).

9.3 Result counts and median elapsed time (7 runs, ms) for all four populations

Query	Rows 100	Rows 1k	Rows 10k	Rows 100k	ms 100	ms 1k	ms 10k	ms 100k
Q1	18	221	2,039	20,817	0.22	1.06	8.95	94.02
Q2	55	477	4,836	49,338	0.96	4.8	53.06	785
Q3	42	482	4,919	48,922	0.2	1.52	12.53	124.91
Q4	32	258	2,513	24,822	0.13	0.76	6.2	71.12
Q5	110	1,200	12,000	120,000	0.59	5.38	59.47	724.92
Q6	0	0	0	0	0.21	1.09	11.09	92.19
Q7	1	5	42	417	0.06	0.12	0.33	2.4
Q8	0	0	0	0	0.45	2.99	28.22	391.12

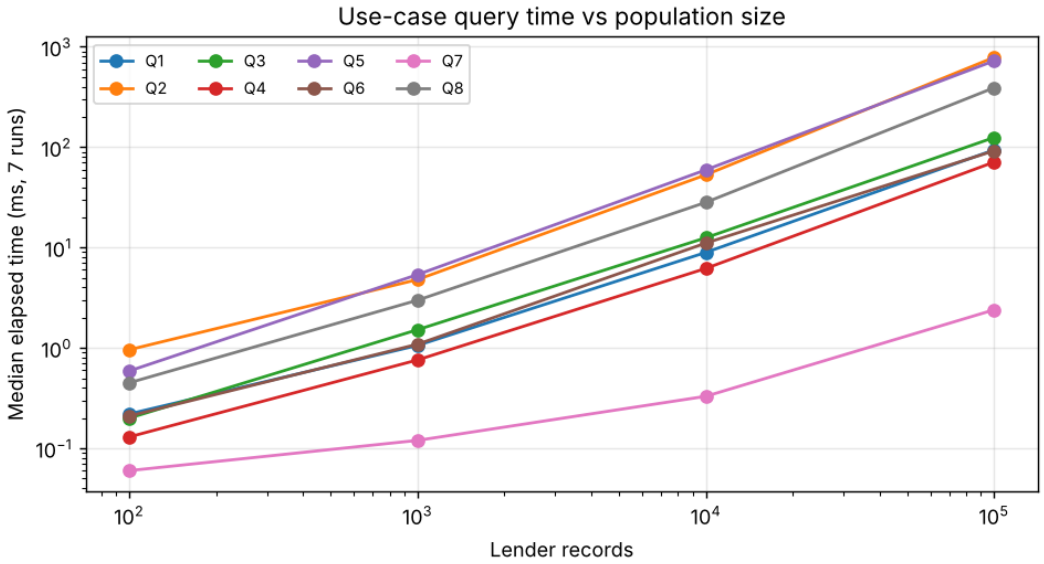


Figure 6 – Median elapsed time per query against lender records (log–log)

Explicit JOIN counts (measured)

Join	Rows 100	Rows 1k	Rows 10k	Rows 100k	ms @100k
loan JOIN lender	106	986	10,046	99,747	76.76
loan JOIN loan_object	249	2,435	24,724	246,623	180.65
loan_object JOIN artifact	249	2,435	24,724	246,623	231.99
loan_object JOIN condition_report	294	2,941	29,690	292,860	407.31
placement JOIN display_slot (composite)	110	1,200	12,000	120,000	137.73

Reading the results. Join row counts equal the child-table counts (loan JOIN lender = 106 = loan; loan_object JOIN condition_report = 294 = condition_report at N = 100), which confirms that the foreign keys neither lose nor duplicate rows. Q2 and Q5 are the slowest at 100,000 lenders (about 0.8 s) because they join 250,000 to 750,000 rows; Q7 stays in the low milliseconds. Growth from 10,000 to 100,000 is about 7×–15× across the queries, so the behaviour is close to linear with this indexing. These timings are from one server in a sandbox with warm caches and should not be used as absolute performance claims.

9.4 Flat LOAN file anomalies (measured on a flat table built from meridian_100)

A flat_loan table was created by joining loan, person, loan_object and artifact (249 rows – one per loan line – repeating lender name, e-mail and loan data on every row).

Anomaly	What happened	Custody / condition meaning
Update	Changing the lender e-mail on one row of a 4-row loan left 2 different e-mail values for the same lender on the same loan.	Notifications about expiry could go to the old or the new address.
Deletion	loan 8 had a single line; deleting it removed lender 'Reddy' from the flat file entirely.	Deleting the only line also deletes the lender and loan facts, and with them the custody history.
Insertion	A new lender with no loan yet cannot be inserted into flat_loan without NULL loan_id/accession_no (loan_id is the row identity).	A newly agreed loan cannot be recorded before its first artifact arrives.

Task 10 – Comparison with File-Based, Hierarchical and Network Models (4 marks)

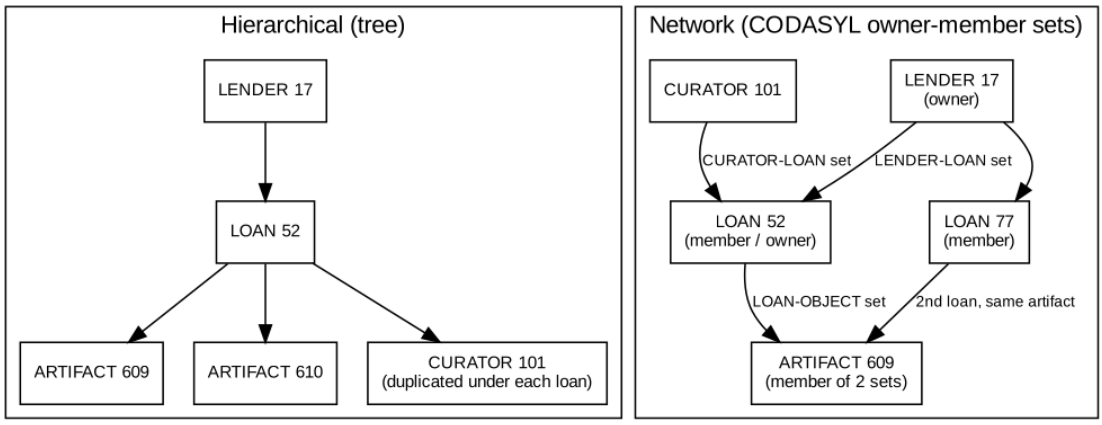


Figure 7 – LENDER–LOAN–CURATOR access paths: tree (left) and network owner–member sets (right)

Aspect	File processing	Hierarchical tree	Network (CODASYL)	Relational (chosen)
Structure	Separate files per department (loan register, gallery plan)	LENDER → LOAN → ARTIFACT / CURATOR	Owner–member sets LENDER-LOAN, LOAN-OBJECT, CURATOR-LOAN	Tables linked by keys
A person in several loans	Name copied into each file	Person repeated under each loan (or a pointer)	One LENDER record owns many LOAN members	One PERSON row; many LOAN rows
Artifact on two loans	Two catalog copies, which can differ	Needs duplication (single parent rule)	Artifact is a member of two sets – supported	ARTIFACT referenced by two LOAN_OBJECT rows
Access path flexibility	Program-bound	Fixed top-down	Navigate set by set	Any join, declarative

Tracing two case queries

- **Q1 (overdue open loans).** Tree: scan every LENDER root and its LOAN children, testing status and expiry. Network: walk each LENDER's LOAN set. File: read the loan register sequentially. Relational: one indexed predicate on ix_loan_status_exp, returning 18 loans at N = 100 without knowing the access path.
- **Q5 (exhibition inventory through placements).** Tree: EXHIBITION is not an ancestor of ARTIFACT, so the tree must be re-rooted or duplicated, and an artifact on two loans exists twice. Network: EXHIBITION owns DISPLAY_SLOTs; each slot is owner of a placement set pointing to the LOAN_OBJECT member, then up to ARTIFACT – feasible but program-navigated. Relational: four-table join (measured 110 rows at N = 100).

Three-schema architecture for Meridian Museum

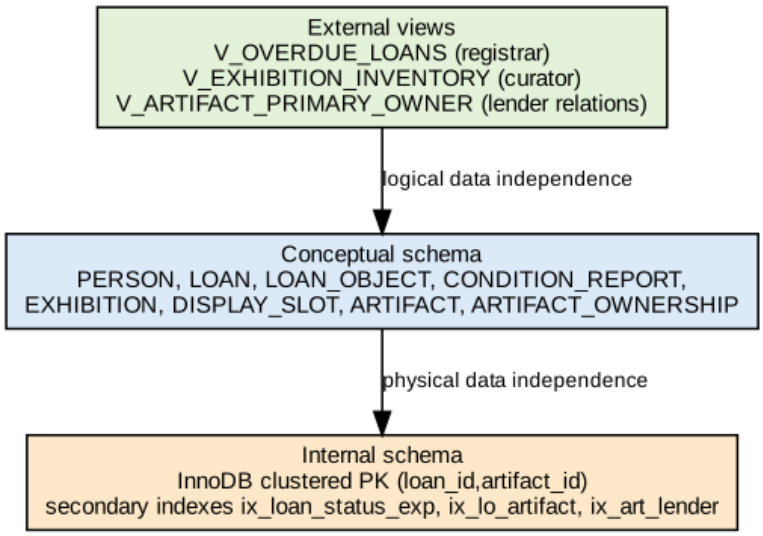


Figure 8 – Operational view, shared conceptual model and indexed internal representation

- **Operational view:** V_ARTIFACT_PRIMARY_OWNER (implemented in the DDL) shows registrars one owner per artifact; an overdue-loans view would show only Q1's columns.
- **Conceptual model:** the 15 relations above, shared by all users.
- **Internal representation:** InnoDB clustered primary keys, plus secondary indexes ix_loan_lender, ix_loan_status_exp, ix_lo_artifact and ix_art_lender; changing an index does not alter any query text.

Components coordinating a concurrent update of the same loan record. The query processor parses the update, the transaction manager assigns the transaction, the lock manager takes a row lock on the loan row (InnoDB), the log/recovery manager writes the undo and redo records, and the buffer manager serves the page. A second registrar updating the same loan waits for the lock and then sees the committed value, so two edits of expiry_date or custody_to cannot silently overwrite each other. **[asserted – no concurrent session was run.]**

Task 11 – Recommendation and Defence of the Final Design (4 marks)

Recommendation: Alternative B (independent LOAN with LOAN_OBJECT custody lines), with the PERSON generalization from Alternative C.

Why it best preserves custody periods and condition evidence

- Each custody period is a row (loan_object) with its own dates; Alternative A has no row to hold it, and Alternative C ties custody to an exhibition rather than to the object.
- Each condition report is attached by composite foreign key to (loan_id, artifact_id). A report filed against "Reddy" is impossible; the design forces filing by physical object on a specific loan.

Traceability

Requirement	Entity / key / association	Evidence
Loan belongs to one lender	LOAN.lender_id NOT NULL FK	Error 1452 when lender 99999 used (Task 8)
Loan contains many artifacts; artifact has successive loans	LOAN_OBJECT (loan_id, artifact_id)	Duplicate line rejected (1062); loans with 3+ lines found (Q3: 42 at N = 100)
Evidence by physical object	CONDITION_REPORT FK to LOAN_OBJECT	loan_object JOIN condition_report = 294 = report rows

Requirement	Entity / key / association	Evidence
Slots numbered within exhibition	DISPLAY_SLOT (exhibition_id, slot_no)	Slot for exhibition 9999 rejected (1452)
Curator may lend	PERSON + LENDER + CURATOR	Q7: 1, 5, 42, 417 persons with both roles

Tested results versus untested assertions

- **Tested (measured):** key, FK and CHECK rejections; row counts and join counts at four sizes; empty-result queries Q6/Q8; the two cross-table rules that were accepted by the engine.
- **Untested:** concurrent registration, triggers for the “at least one line” and “no overlapping custody” rules, the union category implementation, and any index-tuning comparison.

Limitations that remain

- Cross-table rules (shares total 100, placement within loan, one open custody per artifact) are not enforced by the schema; they depend on application code or triggers and are currently only detected by queries.
- ARTIFACT.current_lender_id is kept as the consigning party; if two people share the object, which one is the loan’s lender is a business assumption.
- Identity assumption: two persons with the same name and e-mail-less records are treated as different people.

Growth to 100,000 lenders

At N = 100,000 the database holds 750,000 artifacts, 246,623 loan lines and 292,860 condition reports. Q2 (artifacts without evidence) returned 49,338 rows in about 785 ms and Q5 returned 120,000 rows in about 725 ms; both are acceptable for reporting but would be slow for per-click screens, so they would need pagination or a summary table. Q1 (overdue loans) used the status/expiry index and returned 20,817 loans in about 94 ms. A query that lists missing evidence for one lender would be much faster through ix_loan_lender. **[the claim about the per-lender query is asserted, not measured.]**

Defence by evidence, not by number of relations: the 15 relations exist because each business fact (ownership, custody, placement, evidence) changes on its own schedule. Adding a relation is justified only when a case rule (and a failure case) needs it.

Task 12 – Re-evaluate and Adapt the Design (8 marks)

Changed requirement: artifacts may have shared legal ownership, ownership shares can change while an existing loan is in force, and the custody record must remain distinct from ownership history.

Decision: MODIFY. Keep LOAN, LOAN_OBJECT, CONDITION_REPORT, DISPLAY_SLOT and PLACEMENT unchanged and add a dated ARTIFACT_OWNERSHIP association. Do not redesign.

A. Impact analysis (2 marks)

- **Invalidated assumption:** “each artifact has one current lender” was being used as if the lender were the owner. With shared ownership, ownership is many-to-many over time and the loan’s lender is merely the party that signed the loan.
- **Affected constructs:** ARTIFACT (current_lender_id was the only ownership fact), the relationship ARTIFACT–LENDER (1:N becomes M:N for ownership), business key (artifact_id alone no longer identifies a share – the new fact needs artifact_id + owner_id + valid_from), temporal rules (shares valid_from/valid_to must not overlap per owner and total 100 % at any time).
- **Why the old identifier alone no longer describes the fact:** artifact 609 owned 60 % by A and 40 % by B needs two facts, each varying in time; “artifact_id = 609” cannot say who owns what,

nor from when. Custody is unaffected: loan line (loan_id, artifact_id) still identifies the borrowed object.

- **Trace through the three schema levels:** operational view – V_ARTIFACT_PRIMARY_OWNER keeps the single-owner interface; conceptual – new relation artifact_ownership; internal – a primary key (artifact_id, owner_id, valid_from) and current rows filtered by valid_to IS NULL.

B. Evaluation of the original design (2 marks)

- **EXHIBITION representation:** exhibitions, slots and placements do not mention ownership, so they remain valid; a placement is about custody and time-boxed display, not ownership. **Test [asserted]:** if artifact 609 changes from 100 % to 60/40 mid-loan, no exhibition, slot or placement row needs to change; the measured custody test in part D confirms this for the loan line.
- **Stable keys versus constraints that must change:** person_id, artifact_id, loan_id, exhibition_id and the weak key (exhibition_id, slot_no) are stable. What changes is the constraint “one current lender per artifact” – it stays only as the consignor on ARTIFACT but is no longer treated as the owner.
- **Rejected alternatives revisited: A** (dated relationship) could hold ownership dates but has no loan identity, so ownership changes during a loan would be mixed with custody dates – the exact confusion to avoid. **C** (ternary) has no place for a fourth party or time-varying shares without becoming a quaternary relation.

C. Adaptation (2 marks)

- **Revised EER construct:** ARTIFACT_OWNERSHIP is an associative entity between ARTIFACT and PERSON with attributes share_pct, valid_from and valid_to; a person here can be a lender, curator or institution because PERSON is the generalization (Figure 4).
- **Association replaced/extended/versioned:** the ownership meaning of ARTIFACT–LENDER is **replaced** by the versioned ARTIFACT–OWNERSHIP–PERSON association; the custody association LOAN–LOAN_OBJECT–ARTIFACT is **retained** as is.
- **Migration preserving evidence:** for every artifact, insert one ownership row (artifact_id, current_lender_id, 2020-01-01, NULL, 100); do not touch LOAN, LOAN_OBJECT, CONDITION_REPORT or PLACEMENT. Shared artifacts are then edited by closing the old row (valid_to = change date) and inserting new rows; history is never overwritten.

```
INSERT INTO artifact_ownership (artifact_id, owner_id, valid_from, valid_to, share_pct)
SELECT artifact_id, current_lender_id, '2020-01-01', NULL, 100 FROM artifact;
```

D. Evidence-based defence (2 marks)

- **New identity and participation rules:** identity (artifact_id, owner_id, valid_from); each artifact must have at least one current owner (min 1); a person may own zero or many artifacts (0,N). Measured: share_pct > 100 rejected (ck_own_share); shares totalling 130 were accepted, so the total-100 rule is checked by Q8, not by the schema.
- **Query to reveal an incorrect migration:**

```
SELECT COUNT(*) FROM artifact a WHERE NOT EXISTS (
  SELECT 1 FROM artifact_ownership o
  WHERE o.artifact_id=a.artifact_id AND o.owner_id=a.current_lender_id AND o.valid_to IS NULL);
```

Measured result on N = 100: **0** artifacts without a current ownership row for their lender (a correct migration). A non-zero result would mean lost or mis-attributed ownership. Together with Q8 (share totals) it covers both missing and over-allocated ownership.

- **Effect on generated data and relationship counts:** at N = 100 the 750 artifacts now carry 1440 ownership rows (980 current, 230 artifacts with history). Across sizes (artifacts / total ownership rows / current rows): 750 / 1,440 / 980; 7,500 / 14,157 / 9,719; 75,000 / 142,716 / 97,572; 750,000 / 1,424,334 / 974,778. Ownership therefore adds about 1.9 rows per artifact and does not change the counts of LOAN, LOAN_OBJECT or CONDITION_REPORT.

- **Operational views preserve their interface:** V_ARTIFACT_PRIMARY_OWNER returns one current owner per artifact (the largest share). Measured: 750 view rows for 750 artifacts and 0 ties, so old single-owner reports still return one row per artifact. **Limitation:** an exact 50/50 split would produce two rows; the view would then need a tie-break rule.
- **Demonstration that custody stays distinct from ownership:** for open loan line (52, 609) the current ownership rows were closed on 2026-11-05 mid-loan, and the loan_object row (custody_from, custody_to, insured_value) was compared before and after: unchanged = **true**. The ownership update was then reverted.
- **Cost of extra association and history checks:** one more table, about 1.9 ownership rows per artifact (1.42 million rows at N = 100,000), and extra checks for share totals (Q8 took about 391 ms at 100,000 lenders) and overlapping validity periods. The cost is paid only by ownership-aware queries; custody queries (Q1–Q7) are unchanged.
- **Final decision related to the changed requirement:** the requirement says ownership shares change while a loan stays in force and custody must stay distinct. The measured custody invariance and the unchanged loan, loan-line, report and placement tables show that adding a dated ownership association satisfies it without redesigning the loan model.

Appendix A – DDL used for the implementation (schema.sql)

```
-- Meridian Museum loan coordination schema (preferred design: independently identified LOAN)
-- Includes Task 12 extension: ARTIFACT_OWNERSHIP (shared, dated ownership shares)
SET FOREIGN_KEY_CHECKS=0;
DROP TABLE IF EXISTS artifact_ownership, condition_report, placement, display_slot, exhibition,
  loan_object, loan, artifact, collection_unit, person_phone, registrar, curator, lender, person,
  collection_type, notification_contact;
SET FOREIGN_KEY_CHECKS=1;

CREATE TABLE person (
  person_id   INT AUTO_INCREMENT PRIMARY KEY,
  first_name  VARCHAR(40) NOT NULL,
  last_name   VARCHAR(40) NOT NULL,
  email       VARCHAR(80) NULL,
  UNIQUE KEY uq_person_email (email)
) ENGINE=InnoDB;

CREATE TABLE person_phone (
  person_id INT NOT NULL,
  phone_no  VARCHAR(20) NOT NULL,
  PRIMARY KEY (person_id, phone_no),
  CONSTRAINT fk phone person FOREIGN KEY (person id) REFERENCES person(person id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- PERSON roles (overlapping, partial specialization -> one table per role sharing PK)
CREATE TABLE lender (
  person_id INT PRIMARY KEY,
  lender type ENUM('INDIVIDUAL','INSTITUTION') NOT NULL DEFAULT 'INDIVIDUAL',
  CONSTRAINT fk_lender_person FOREIGN KEY (person_id) REFERENCES person(person_id)
) ENGINE=InnoDB;

CREATE TABLE collection_unit (
  unit id INT AUTO INCREMENT PRIMARY KEY,
  unit_name VARCHAR(60) NOT NULL,
  UNIQUE KEY uq_unit_name (unit_name)
) ENGINE=InnoDB;

CREATE TABLE curator (
  person_id INT PRIMARY KEY,
  unit_id INT NOT NULL,
  hire_date DATE NOT NULL,
  CONSTRAINT fk_curator_person FOREIGN KEY (person_id) REFERENCES person(person_id),
  CONSTRAINT fk_curator_unit FOREIGN KEY (unit_id) REFERENCES collection_unit(unit_id)
) ENGINE=InnoDB;

CREATE TABLE registrar (
  person_id INT PRIMARY KEY,
  certification VARCHAR(40) NULL,
  CONSTRAINT fk_registrar_person FOREIGN KEY (person_id) REFERENCES person(person_id)
) ENGINE=InnoDB;

CREATE TABLE collection_type (
  type_code VARCHAR(10) PRIMARY KEY,
  type_name VARCHAR(40) NOT NULL,
  handling_note VARCHAR(120) NULL
) ENGINE=InnoDB;

CREATE TABLE artifact (
  artifact_id INT AUTO_INCREMENT PRIMARY KEY,
  accession_no VARCHAR(20) NOT NULL,
  catalog_title VARCHAR(120) NOT NULL,
  type_code VARCHAR(10) NOT NULL,
  unit_id INT NOT NULL,
  current_lender_id INT NOT NULL,           -- each artifact has one current lender
  UNIQUE KEY uq_accession (accession_no),
  CONSTRAINT fk_art_type FOREIGN KEY (type_code) REFERENCES collection_type(type_code),
  CONSTRAINT fk_art_unit FOREIGN KEY (unit_id) REFERENCES collection_unit(unit_id),
  CONSTRAINT fk_art_lender FOREIGN KEY (current_lender_id) REFERENCES lender(person_id)
) ENGINE=InnoDB;

CREATE TABLE loan (
  loan_id INT AUTO_INCREMENT PRIMARY KEY,
  loan_ref VARCHAR(20) NOT NULL,
  lender_id INT NOT NULL,                   -- a loan belongs to one lender
  registrar_id INT NOT NULL,                -- operational user who creates the record
```

```
  start_date DATE NOT NULL,
  expiry_date DATE NOT NULL,
  return_date DATE NULL,
  status ENUM('OPEN','RETURNED','CANCELLED') NOT NULL DEFAULT 'OPEN',
  UNIQUE KEY uq_loan_ref (loan_ref),
  CONSTRAINT ck_loan_period CHECK (expiry_date > start_date),
  CONSTRAINT ck_loan_return CHECK (return_date IS NULL OR return_date >= start_date),
  CONSTRAINT fk_loan_lender FOREIGN KEY (lender_id) REFERENCES lender(person_id),
  CONSTRAINT fk_loan_reg FOREIGN KEY (registrar_id) REFERENCES registrar(person_id)
) ENGINE=InnoDB;

-- M:N between LOAN and ARTIFACT with its own facts (insurance value, custody dates)
CREATE TABLE loan_object (
  loan_id INT NOT NULL,
  artifact_id INT NOT NULL,
  insured_value DECIMAL(12,2) NOT NULL,
  custody_from DATE NOT NULL,
  custody_to DATE NULL,
  PRIMARY KEY (loan_id, artifact_id),
  CONSTRAINT ck_lo_value CHECK (insured_value >= 0),
  CONSTRAINT ck_lo_custody CHECK (custody_to IS NULL OR custody_to >= custody_from),
  CONSTRAINT fk_lo_loan FOREIGN KEY (loan_id) REFERENCES loan(loan_id),
  CONSTRAINT fk_lo_art FOREIGN KEY (artifact_id) REFERENCES artifact(artifact_id)
) ENGINE=InnoDB;

-- condition evidence belongs to the physical object within a loan line, not to the lender name
CREATE TABLE condition_report (
  loan_id INT NOT NULL,
  artifact_id INT NOT NULL,
  report_no INT NOT NULL,
  report date DATE NOT NULL,
  stage ENUM('INCOMING','DISPLAY','OUTGOING') NOT NULL,
  rating ENUM('EXCELLENT','GOOD','FAIR','POOR') NOT NULL,
  curator_id INT NOT NULL,
  PRIMARY KEY (loan_id, artifact_id, report_no),
  CONSTRAINT fk_cr_lo FOREIGN KEY (loan_id, artifact_id) REFERENCES loan_object(loan_id,
artifact id),
  CONSTRAINT fk_cr_cur FOREIGN KEY (curator_id) REFERENCES curator(person_id)
) ENGINE=InnoDB;

CREATE TABLE exhibition (
  exhibition_id INT AUTO_INCREMENT PRIMARY KEY,
  exhibition_name VARCHAR(80) NOT NULL,     -- NOT unique: titles are reused
  open_date DATE NOT NULL,
  close_date DATE NOT NULL,
  curator_id INT NOT NULL,
  CONSTRAINT ck_ex_period CHECK (close_date > open_date),
  CONSTRAINT fk_ex_cur FOREIGN KEY (curator_id) REFERENCES curator(person_id)
) ENGINE=InnoDB;

-- weak entity: identified by (exhibition_id, slot_no)
CREATE TABLE display_slot (
  exhibition_id INT NOT NULL,
  slot_no INT NOT NULL,
  gallery VARCHAR(30) NOT NULL,
  PRIMARY KEY (exhibition_id, slot_no),
  CONSTRAINT ck_slot_no CHECK (slot_no > 0),
  CONSTRAINT fk_slot_ex FOREIGN KEY (exhibition_id) REFERENCES exhibition(exhibition_id) ON DELETE
CASCADE
) ENGINE=InnoDB;

-- placement: a loan line is placed in a slot (composite FK to slot)
CREATE TABLE placement (
  loan_id INT NOT NULL,
  artifact_id INT NOT NULL,
  exhibition_id INT NOT NULL,
  slot_no INT NOT NULL,
  placed_from DATE NOT NULL,
  placed_to DATE NOT NULL,
  PRIMARY KEY (loan_id, artifact_id),
  UNIQUE KEY uq_slot_time (exhibition_id, slot_no, placed_from),
  CONSTRAINT ck_pl_period CHECK (placed_to >= placed_from),
  CONSTRAINT fk_pl_lo FOREIGN KEY (loan_id, artifact_id) REFERENCES loan_object(loan_id,
artifact_id),
  CONSTRAINT fk_pl_slot FOREIGN KEY (exhibition_id, slot_no) REFERENCES display_slot(exhibition_id,
slot_no)
) ENGINE=InnoDB;
```

```
-- Task 12: shared, dated legal ownership (kept separate from custody = loan_object)
CREATE TABLE artifact_ownership (
  artifact_id INT NOT NULL,
  owner_id INT NOT NULL,
  valid_from DATE NOT NULL,
  valid_to DATE NULL,                      -- NULL = current share
  share_pct DECIMAL(5,2) NOT NULL,
  PRIMARY KEY (artifact_id, owner_id, valid_from),
  CONSTRAINT ck_own_share CHECK (share_pct > 0 AND share_pct <= 100),
  CONSTRAINT ck_own_period CHECK (valid_to IS NULL OR valid_to > valid_from),
  CONSTRAINT fk_own_art FOREIGN KEY (artifact_id) REFERENCES artifact(artifact_id),
  CONSTRAINT fk_own_person FOREIGN KEY (owner_id) REFERENCES person(person_id)
) ENGINE=InnoDB;

-- Stable interface for operational users (view preserves the old "one lender per artifact" shape)
CREATE OR REPLACE VIEW v_artifact_primary_owner AS
SELECT o.artifact_id, o.owner_id AS primary_owner_id, o.share_pct
FROM artifact_ownership o
WHERE o.valid_to IS NULL
  AND o.share_pct = (SELECT MAX(o2.share_pct) FROM artifact_ownership o2
                     WHERE o2.artifact_id=o.artifact_id AND o2.valid_to IS NULL);

CREATE INDEX ix_loan_lender ON loan(lender_id);
CREATE INDEX ix_loan_status_exp ON loan(status, expiry_date);
CREATE INDEX ix lo artifact ON loan object(artifact id);
CREATE INDEX ix_art_lender ON artifact(current_lender_id);
```

Deliverable scripts: schema.sql, generate.py (seed 4211) and validate.py are supplied with this report; results.json holds all measured values quoted above.